Έτσι, με τη διολίσθηση χάνεται ένας αριθμός από bits από τη δεξιά πλευρά του αντιγράφου. Τα bits που χάνονται αντικαθίστανται στην αριστερή πλευρά του αντιγράφου με τιμή την τιμή του πρώτου bit του, δηλαδή την τιμή του προσήμου. Ας σημειωθεί εδώ πως η Java χρησιμοποιεί για την κωδικοποίηση ακέραιων την παράσταση μέτρο και πρόσημο για τις θετικές τιμές και την παράσταση συμπληρώματος ως προς δύο (two's complement) για τις αρνητικές. Επομένως, η τιμή του προσήμου είναι 0 για τους θετικούς ακέραιους και 1 για τους αρνητικούς. Για λεπτομέρειες σχετικά με την κωδικοποίηση των ακέραιων σε μορφή συμπληρώματος ως προς δύο, ανατρέξτε στο παράρτημα 1.

```
1   static void rightShift() { //Κώδικας 4.14
2       int i = 3;
3       int j = -3;
4       System.out.println("Integer.toBinaryString(" + i + ")=" +
Integer.toBinaryString(i));
5       System.out.println("Integer.toBinaryString(" + i + ">>1)=" +
Integer.toBinaryString(i >> 1));
6       System.out.println("Integer.toBinaryString(" + j + ")=" +
Integer.toBinaryString(j));
7       System.out.println("Integer.toBinaryString(" + j + ">>1)=" +
Integer.toBinaryString(j >> 1));
8   }
```

Κώδικας 4.14*: Ο τελεστής δεξιάς διολίσθησης*

Στον κώδικα 4.14 έχουμε ορίσει τις ακέραιες μεταβλητές i και j. Στη γραμμή 4 εκτυπώνεται ο ψηφιοχάρτης της μεταβλητής i με τη βοήθεια της συνάρτησης toBinaryString. Η έξοδος αυτής της γραμμής είναι Integer.toBinaryString(3)=11. Πράγματι, η αναπαράσταση του 3 με τη μορφή συμπληρώματος του δύο είναι 11. Βέβαια, δεδομένου, ότι ο int στην Java έχει μέγεθος 32 bit, η πραγματική αναπαράσταση στη μνήμη είναι 00000000000000000000000000000011. Απλώς η Java δεν τυπώνει τυχόν μηδενικά στην αρχή ενός αριθμού (leading zeroes) δεδομένου ότι αυτά δεν επηρεάζουν την τιμή του. Η γραμμή 5 εκτυπώνει Integer.toBinaryString(3>>1)=1. Στη μεταβλητή i έγινε διολίσθηση δεξιά κατά 1 bit, επομένως από το αποτέλεσμα χάθηκε το τελευταίο 1, ενώ παράλληλα προστέθηκε στην αρχή ένα μηδέν. Ο πλήρης ψηφιοχάρτης του αποτελέσματος είναι 00000000000000000000000000000001.

Στη γραμμή 6 τυπώνεται ο ψηφιοχάρτης του -3. Πιο συγκεκριμένα, η έξοδος της γραμμής 6 είναι Integer.toBinaryString(-3)=11111111111111111111111111111101. Μετά τη διολίσθηση δεξιά κατά 1, το τελευταίο 1 χάνεται και προστίθεται στην αρχή του αποτελέσματος το ψηφίο 1. Έτσι, η έξοδος της γραμμής 7 είναι Integer.toBinaryString(-3>>1)=11111111111111111111111111111110. Ο ψηφιοχάρτης αυτός αντιστοιχεί στη δεκαδική τιμή -2.

4.1.7.2 Διολίσθηση αριστερά

Ο τελεστής αριστερής διολίσθησης (left shift) είναι δυαδικός, συμβολίζεται ως "<<"και συντάσσεται με ακέραιους τύπους. Η διαφορά του από τη διολίσθηση δεξιά είναι πως τα bits χάνονται από αριστερά και προστίθενται δεξιά. Επιπλέον, τα bits που προστίθενται έχουν την τιμή μηδέν ανεξάρτητα από το πρόσημο του πρώτου τελεστέου. Κατά τα λοιπά, ισχύουν κατ' αναλογία όσα ισχύουν και για τον τελεστή δεξιάς διολίσθησης.

```
    static void leftShift() { //Κώδικας 4.15
        int i = 3;
        int j = -3;
        System.out.println("Integer.toBinaryString(" + i + ")=       " +
Integer.toBinaryString(i));
        System.out.println("Integer.toBinaryString(" + i + "<<1)=" +
Integer.toBinaryString(i << 1));
        System.out.println("Integer.toBinaryString(" + j + ")=" +
Integer.toBinaryString(j));
```

```
        System.out.println("Integer.toBinaryString(" + j + "<<1)=" +
Integer.toBinaryString(j << 1));
        System.out.println("value of j<<1=" + (j << 1));
    }
```

Κώδικας 4.15*: Τελεστής αριστερής διολίσθησης*

Η έξοδος του κώδικα 4.15 είναι η ακόλουθη:

```
Integer.toBinaryString(3)=11
Integer.toBinaryString(3<<1)= 110
Integer.toBinaryString(-3)=11111111111111111111111111111101
Integer.toBinaryString(-3<<1)= 11111111111111111111111111111010
value of j<<1=-6
```

Προσέξτε τον ψηφιοχάρτη του j, δηλαδή του -3. Μετά τη διολίσθηση έχει χαθεί ένα ψηφίο με τιμή 1 ενώ στο τέλος του ψηφιοχάρτη έχει προστεθεί ένα ψηφίο με τιμή 0. Η τιμή του j μετά τη διολίσθηση έχει γίνει -6.

4.1.7.3 Μη προσημασμένη διολίσθηση δεξιά

Ο τελεστής μη προσημασμένης δεξιάς διολίσθησης (unsigned right shift) συμβολίζεται με >>> και διαφέρει από τον τελεστή δεξιάς διολίσθησης κατά το ότι τα ψηφία που αντικαθιστούν όσα χάνονται δεν εξαρτώνται από το πρόσημο του πρώτου τελεστέου αλλά έχουν πάντα την τιμή 0.

Ας σημειωθεί πως στην Java δεν υπάρχει τελεστής μη προσημασμένης αριστερής διολίσθησης, καθώς αυτός θα ήταν πανομοιότυπος με την αριστερή διολίσθηση.

4.1.8 Σύνθετη εκχώρηση

Υποστηρίζεται μια σειρά δυαδικών τελεστών σύνθετης εκχώρησης (compound assignment), δηλαδή τελεστών που συνδυάζουν την εκχώρηση με μία άλλη πράξη. Στον πίνακα 4.5 παρουσιάζονται οι τελεστές σύνθετης εκχώρησης μαζί με την ισοδύναμη έκφραση που αντιστοιχεί στον κάθε ένα.

Τελεστής	**Παράδειγμα Χρήσης**	**Ισοδύναμη έκφραση**
+=	i+=5	i=i+5
-=	i-=k	i=i-k
*=	i*=k	i=i*k
/=	i/=k	i=i/k
%=	i%=k	i=i%k
&=	i&=k	i=i&k
|=	i|=k	i=i|k
<<=	i<<=k	i=i<<k
>>=	i>>=k	i=i>>k
>>>=	i>>>=k	i=i>>>k

Πίνακας 4.5 *Τελεστές Σύνθετης Εκχώρησης*

4.2. Βασική Διαμόρφωση εξόδου

Συχνά χρειάζεται να διαμορφώσουμε τα δεδομένα πριν τα εμφανίσουμε. Για παράδειγμα, σε μια εφαρμογή μπορεί να επιθυμούμε να εμφανίσουμε τους πραγματικούς αριθμούς με δύο δεκαδικά ψηφία. Σε αυτήν την ενότητα παρουσιάζονται δύο εναλλακτικοί τρόποι για τη διαμόρφωση της εξόδου.

4.2.1 Η printf

Η printf αποτελεί τον παραδοσιακό τρόπο διαμόρφωσης της εξόδου με την έννοια ότι υποστηρίζεται και στην C [2]. Ας ξεκινήσουμε την παρουσίαση με ένα παράδειγμα:

```
1   static void stringF() { ////Κώδικας 4.16
2       System.out.printf("%s%n", "Hello");
3       System.out.printf("%-10s%10S%n", "Hello", "Hello");
4       System.out.printf("%10.3S%n", "Hello");
5   }
```

***Κώδικας 4.16**: Διαμόρφωση εξόδου του τύπου String*

Η πρώτη παράμετρος σε όλες τις κλήσεις της printf στον κώδικα 4.16 ονομάζεται προσδιοριστής διαμόρφωσης (format specifier). Ο προσδιοριστής διαμόρφωσης περιλαμβάνει μια σειρά από κανόνες διαμόρφωσης (formatting rules). Κάθε κανόνας σηματοδοτεί την αρχή του με τον χαρακτήρα %. Στη γραμμή 2, ο προσδιοριστής διαμόρφωσης έχει την τιμή %s%n. Επομένως περιλαμβάνει δύο κανόνες: Ο πρώτος κανόνας εκφράζεται με τον χαρακτήρα s και μας λέει πως η διαμόρφωση αφορά αλφαριθμητική σειρά. Ο χαρακτήρας s σε αυτό το πλαίσιο ονομάζεται χαρακτήρας μετατροπής (conversion character). Άλλοι χαρακτήρες μετατροπής είναι ο d για διαμόρφωση ακέραιων, ο f για διαμόρφωση πραγματικών αριθμών, ο c για διαμόρφωση χαρακτήρων και ο b για διαμόρφωση τιμών boolean. Ωστόσο, οι χαρακτήρες μετατροπής c και b χρησιμοποιούνται σπάνια. Ο δεύτερος κανόνας περιλαμβάνει τον χαρακτήρα n που προκαλεί αλλαγή γραμμής.

Η δεύτερη παράμετρος στη γραμμή 2 είναι το String που επιθυμούμε να τυπώσουμε. Καθότι ο κανόνας διαμόρφωσης στην πρώτη γραμμή δεν καθορίζει τίποτα περισσότερο παρά μόνο ότι θα εμφανίσουμε μια αλφαριθμητική σειρά και θα προβούμε σε αλλαγή γραμμής, εκείνο που θα δούμε ως έξοδο είναι απλώς η σειρά Hello.

Στη γραμμή 3, μετά τον προσδιοριστή διαμόρφωσης έχουμε μια σειρά από String. Είναι γενικό χαρακτηριστικό της printf ότι μετά τον προσδιοριστή διαμόρφωσης μπορεί να δέχεται πολλές παραμέτρους. Για περισσότερες πληροφορίες επάνω σε αυτό, δείτε την ενότητα 7.3.3. Κάθε μία από αυτές τις παραμέτρους προορίζεται για να εμφανιστεί. Τη διαμόρφωσή τους ρυθμίζει ο προσδιοριστής διαμόρφωσης, δηλαδή η πρώτη παράμετρος. Στη γραμμή 3 λοιπόν έχουμε αυτόν τον προσδιοριστή διαμόρφωσης %-10s%10S%n. Περιλαμβάνει τρεις κανόνες. Ο τρίτος, όπως εξηγήθηκε παραπάνω, προκαλεί αλλαγή γραμμής της εξόδου. Ο πρώτος αφορά τη δεύτερη παράμετρο της printf και ο δεύτερος την τρίτη. Οι δύο πρώτοι κανόνες περιλαμβάνουν το 10. Αυτό προσδιορίζει το πλάτος στο οποίο θα εμφανιστεί η αντίστοιχη παράμετρος. Τα δεδομένα στοιχίζονται σε αυτό το πλάτος στη δεξιά πλευρά, δηλαδή το Hello σε πλάτος 10 θα εμφανιστεί ως ␣␣␣␣␣Hello, που ␣ συμβολίζει ένα διάστημα (space). Αν ωστόσο επιθυμούμε τα δεδομένα μας να στοιχηθούν στα αριστερά του πλάτους, τότε τοποθετούμε τον χαρακτήρα – (μείον) όπως έχουμε κάνει στον πρώτο κανόνα. Σημειώστε πως σε αυτό το πλαίσιο ο χαρακτήρας μείον ονομάζεται σηματοφόρος (flag). Έτσι το πρώτο Hello θα εμφανισθεί ως Hello ␣␣␣␣␣. Αν επιθυμούμε η σειρά να εμφανιστεί με κεφαλαία, τότε ως χαρακτήρα διαμόρφωσης δίνουμε το S αντί του s, όπως κάνουμε στον δεύτερο κανόνα. Στη γραμμή 4 το πλάτος ακολουθείται από την ακρίβεια (precision). Η ακρίβεια δηλώνεται με μια τελεία ακολουθούμενη από έναν ακέραιο.
Η έξοδος του κώδικα 4.16 έχει ως εξής:

```
Hello
Hello          HELLO
       HEL
```

Το γενικό συντακτικό ενός κανόνα διαμόρφωσης έχει ως εξής:

```
%[σηματοφορείς][πλάτος][.ακρίβεια]χαρακτήρας μετατροπής
```

Από αυτά τα στοιχεία, μόνο ο χαρακτήρας % που δηλώνει την αρχή του κανόνα και ο χαρακτήρας μετατροπής που δηλώνει τον τύπο των δεδομένων ή αλλαγή γραμμής είναι υποχρεωτικά. Οι σηματοφορείς, το πλάτος και η ακρίβεια είναι προαιρετικά στοιχεία. Ωστόσο, η σειρά με την οποία τα στοιχεία τοποθετούνται μέσα σε έναν κανόνα είναι υποχρεωτικά και αυτή δείχνει το γενικό συντακτικό του κανόνα. Ας δούμε και μερικά άλλα χρήσιμα παραδείγματα:

```
1   static void intF() { //Κώδικας 4.17
2       System.out.printf("ακέραιος: %d%n", 1000000L);
3       System.out.printf("ακέραιος: %,d%n", 1000000L);
4       System.out.printf(Locale.ENGLISH,"ακέραιος: %,d%n", 1000000L);
5   }
```

***Κώδικας 4.17**: Διαμόρφωση εξόδου ακέραιων τύπων*

Όπως φαίνεται στον κώδικα 4.17, ένας προσδιοριστής διαμόρφωσης μπορεί να περιλαμβάνει και απλές αλφαριθμητικές σειρές οι οποίες απλώς αποτελούν μέρος της εξόδου. Στη γραμμή 2 δεν δίνουμε κάποια ιδιαίτερη διαμόρφωση, οπότε η έξοδος αυτής της γραμμής είναι απλά 1000000. Στη γραμμή 3 όμως, έχουμε τοποθετήσει τον σηματοφόρο (κόμμα). Με αυτόν τον σηματοφόρο ζητάμε ο αριθμός να εμφανιστεί με διαχωριστικά χιλιάδων, οπότε η έξοδος εδώ θα είναι 1.000.000. Επειδή όμως τα διαχωριστικά χιλιάδων δεν είναι ίδια για όλες τις χώρες, μας δίνεται η δυνατότητα να χρησιμοποιήσουμε μια άλλη έκδοση της printf που μας επιτρέπει να καθορίσουμε ποιο ακριβώς διαχωριστικό θα χρησιμοποιήσουμε. Έτσι, η γραμμή 4 εμφανίζει 1,000,000.

```
1     static void realF() { //Κώδικας 4.18
2       System.out.printf("%f%n", 5.1473);
3       System.out.printf("%(010.2f%n", -5.1473);
4       System.out.printf("%-10.2f%n", 5.1473);
5   }
```

***Κώδικας 4.18**: Διαμόρφωση εξόδου πραγματικών τύπων*

Η έξοδος του κώδικα 4.18 έχει ως εξής:

```
5,147300
(00005,15)
5,15
```

Στη γραμμή 3, ο σηματοφόρος '(' (αριστερή παρένθεση) έχει ως αποτέλεσμα να εμφανίζονται οι αρνητικοί αριθμοί εντός παρενθέσεων. Το δε 0 που ακολουθεί καλύπτει το πλάτος με μηδενικά στα αριστερά. Ο σηματοφόρος 0 δεν συντάσσεται με στοίχιση στα αριστερά. Κατά τα λοιπά, με όσα έχουμε συζητήσει μέχρι εδώ, είναι εύκολο να ερμηνεύσετε την έξοδο του κώδικα 4.18.

4.2.2 Η DecimalFormat

Ένας εναλλακτικός τρόπος για διαμόρφωση δεκαδικών αριθμών για έξοδο βασίζεται στην κλάση DecimalFormat. Η DecimalFormat παρέχει ευκολίες στη διαμόρφωση των αριθμών για έξοδο, εισάγει ωστόσο κάποιον βαθμό πολυπλοκότητας στον κώδικά μας.

```
1    static void outputFormat() { //Κώδικας 4.19
2        DecimalFormat f = new DecimalFormat("###,###.##");
3        double d1 = 9.566, d2 = 9.273;
4        System.out.println(f.format(d1) + "+" + f.format(d2) + " = " +
f.format(d1 + d2));
5
```

```
6         d1 = 9.565;
7         d2 = 9.275;
8         System.out.println(f.format(d1) + "+" + f.format(d2) + " = " +
f.format(d1 + d2));
9
10        d1 = 0.275;
11        d2 = 0.385;
12        System.out.println(f.format(d1) + "+" + f.format(d2) + " = " +
f.format(d1 + d2));
13
14        d1 = 9_856_327.563;
15        d2 = 5627869.289;
16        System.out.println(f.format(d1) + "+" + f.format(d2) + " = " +
f.format(d1 + d2));
17
18        f.applyPattern("###,###.###");
19        System.out.println(f.format(d1) + "+" + f.format(d2) + " = " +
f.format(d1 + d2));
20
21        f.applyPattern("€###,###.###");
22        System.out.println(f.format(d1));

23        f.applyPattern("(€###,###.###)");
24        System.out.println(f.format(d1));
25        int k = 128;
26        System.out.println(f.format(k));
27
28        char c = 'A';
29        System.out.println(f.format(c));
30
31        f.applyPattern("000,000,000.0000");
32        System.out.println(f.format(d1));
33    }
```

***Κώδικας 4.19**: Διαμόρφωση εξόδου*

Η έξοδος της outputFormat είναι η εξής:

```
9,57+9,27 = 18,84
9,56+9,28 = 18,84
0,28+0,39 = 0,66
9.856.327,56+5.627.869,29 = 15.484.196,85
9.856.327,563+5.627.869,289 = 15.484.196,852
€9.856.327,563
(€9.856.327,563)
(€128)
(€65)
009.856.327,5630
```

Η διαμόρφωση βασίζεται στην προκαθορισμένη κλάση DecimalFormat [2]. Για να τη χρησιμοποιήσετε προσθέστε στην κλάση σας την import Java.text.DecimalFormat.

Στη γραμμή 2 του κώδικα 4.19 δημιουργούμε ένα αντικείμενο της κλάσης DecimalFormat. Η μεταβλητή f είναι μεταβλητή αναφοράς. Κατά τη δημιουργία της συνδέεται με το μοτίβο (pattern), "###,###.##". Στη γραμμή 4 στέλνουμε προς εμφάνιση στην οθόνη τρεις πραγματικές τιμές, χρησιμοποιώντας τη συνάρτηση format της DecimalFormat. Η συνάρτηση format επιστρέφει μια

αλφαριθμητική σειρά. Η σειρά που επιστρέφει η format υπολογίζεται με βάση την τιμή που της δίνουμε κατά την κλήση της και το μοτίβο που ορίσαμε κατά τη δημιουργία της f. Στο μοτίβο αυτό ο χαρακτήρας ‘,’ συμβολίζει τον διαχωριστή χιλιάδων, ο χαρακτήρας ‘.’ συμβολίζει τον διαχωριστή δεκαδικών και ο χαρακτήρας ‘#’ συμβολίζει το ψηφίο. Επομένως, σύμφωνα με το μοτίβο, ζητάμε μια διαμόρφωση στην οποία να εμφανίζονται τα ψηφία του αριθμού κατά τρόπο που οι χιλιάδες να διαχωρίζονται μεταξύ τους, όπως επίσης και να διαχωρίζεται το δεκαδικό από το ακέραιο μέρος και το δεκαδικό μέρος να εμφανίζεται με δύο ψηφία. Η έξοδος της γραμμής 4 είναι 9.57+9.27 = 18.84. Προσέξτε καταρχάς πως έχουμε στρογγυλοποίηση του δεκαδικού μέρους. Η μεταβλητή d1 εμφανίστηκε ως 9.57 αντί του 9.566. Αντίστοιχα, η d2 εμφανίζεται ως 9.27 αντί του 9.273. Η format στρογγυλοποιεί εξ ορισμού τα δεκαδικά προς την πλησιέστερη τιμή. Το 0.57 είναι πλησιέστερο στο 0.566 σε σχέση με το 0.56. Παρόμοια, το 0.27 είναι πλησιέστερο στο 0.273 από ότι το 0.28.

Όταν όμως δεν υπάρχει πλησιέστερη τιμή, τότε η στρογγυλοποίηση γίνεται προς τον άρτιο γείτονα. Η έξοδος της γραμμής 8 είναι 9.56+9.28 = 18.84. Προσέξτε πως η τιμή του d1 από 9.565 μετατράπηκε σε 9.56. Πράγματι, η τιμή 9.565 απέχει εξίσου από το 9.56 και το 9.57. Σε αυτήν την περίπτωση η format στρογγυλοποίησε προς τον άρτιο γείτονα, δηλαδή το 6 και όχι τον περιττό, δηλαδή το 7. Αντίστοιχα, η d2 με τιμή 9.275, εμφανίζεται ως 9.28. Επομένως, η στρογγυλοποίηση έγινε προς τον άρτιο γείτονα, το 8, και όχι τον περιττό, δηλαδή το 7.

Η έξοδος της γραμμής 12, μετά τη μεταβολή των τιμών των d1 και d2 που έγινε στη γραμμή 10 και 11, είναι 0.28+0.39 = 0.66. Επομένως, το d1 με τιμή 0.275 εμφανίζεται ως 0.28 και το d2 με τιμή 0.385 εμφανίζεται ως 0.39. Εκείνο όμως που χρήζει ιδιαίτερης προσοχής είναι πως το άθροισμα 0.28+0.39 ισούται με 0.67 και όχι με 0.66. Η ασυμφωνία αυτή προκύπτει καθώς υπολογίζουμε πρώτα το άθροισμα d1+d2 και στη συνέχεια το στρογγυλοποιούμε. Επομένως, 0.275+0.385=0.610. Η στρογγυλοποίηση στα δύο δεκαδικά του 0.610 είναι προφανώς το 0.61. Είναι κατανοητό από αυτό το παράδειγμα πως η στρογγυλοποίηση κατά την εμφάνιση πραγματικών αριθμών πρέπει να γίνεται με ιδιαίτερη προσοχή.

Στη γραμμή 16 το μοτίβο “###,###.##“ εφαρμόζεται σε αριθμούς με περισσότερα ψηφία από τους χαρακτήρες του χωρίς να δημιουργείται κάποιο πρόβλημα.

Στη γραμμή 18 αλλάζουμε το μοτίβο της f. Το νέο μοτίβο υποστηρίζει τρία δεκαδικά ψηφία. Έτσι η έξοδος της γραμμής 19 είναι 9,856,327.563+5,627,869.289 = 15,484,196.852.

Είναι εφικτό κατά τη διαμόρφωση των αριθμών να προσθέτουμε σταθερούς χαρακτήρες. Έτσι στη γραμμή 21 έχουμε προσθέσει το σύμβολο του Ευρώ ώστε να προηγείται της αριθμητικής τιμής. Πράγματι η έξοδος της γραμμής 22 είναι €9,856,327.563.

Στη γραμμή 23 τροποποιούμε πάλι το μοτίβο της f ώστε να περικλείεται ο αριθμός και το σύμβολο του Ευρώ από παρενθέσεις. Έτσι, η έξοδος της γραμμής 24 είναι (€9,856,327.563).

Στη γραμμή 26 επιδεικνύεται πώς το μοτίβο μπορεί να εφαρμοστεί και σε ακέραιους. Η έξοδος της γραμμής είναι (€128). Παρότι το μοτίβο προβλέπει δεκαδικά ψηφία, η format τυπώνει σωστά τους ακέραιους τύπους χωρίς δεκαδικά.

Στη γραμμή 29 εκτυπώνεται ο χαρακτήρας ‘A’ μέσω της format. Η έξοδος της γραμμής είναι (€65), που σημαίνει πως στη θέση του χαρακτήρα εμφανίζεται ο κωδικός του.

Σε κάποιες εφαρμογές θέλουμε να εμφανίζουμε αριθμούς σε σταθερό πλάτος συμπληρώνοντας τα στοιχεία που λείπουν με μηδενικά. Σε αυτήν την περίπτωση, προκειμένου να μην αλλοιωθεί η τιμή του αριθμού, μηδενικά μπορούν να προστεθούν μόνο στην αρχή του ακέραιου μέρους ή στο τέλος του δεκαδικού μέρους ενός αριθμού. Στη γραμμή 31 μεταβάλλουμε το μοτίβο, έτσι ώστε ο αριθμός να τοποθετείται σε πλάτος 13 ψηφίων από τα οποία τα 9 ανήκουν στο ακέραιο μέρος και τα 4 στο δεκαδικό. Η έξοδος της γραμμής 32 είναι 009,856,327.5630.

Ας σημειωθεί πως η κλάση DecimalFormat παρέχει πολλές επιπλέον δυνατότητες. Για παράδειγμα, τη δυνατότητα να καθορίσουμε ως διαχωριστικό χιλιάδων τον χαρακτήρα ‘.’ και ως διαχωριστικό δεκαδικών τον χαρακτήρα ‘,’ ή να αλλάξουμε τον τρόπο στρογγυλοποίησης. Περισσότερες πληροφορίες για την DecimalFormat μπορείτε να βρείτε στην τεκμηρίωση της Oracle [1].

Επιπλέον, υπάρχουν και άλλοι τρόποι για τη διαμόρφωση της εξόδου όπως η συνάρτηση format της κλάσης String [3].

4.3 Είσοδος από το πληκτρολόγιο

Για είσοδο από το πληκτρολόγιο θα χρησιμοποιούμε την προκαθορισμένη κλάση Scanner. Για να χρησιμοποιηθεί πρέπει να την εισάγετε στον κώδικά σας προσθέτοντας την import Java.util.Scanner. Με την Scanner μπορούμε να διαβάσουμε δεδομένα και από αρχεία. Ωστόσο σε αυτήν την ενότητα, περιοριζόμαστε στη χρήση της Scanner για απλή είσοδο από το πληκτρολόγιο. Η ανάγνωση αρχείων παρουσιάζεται αναλυτικά στην ενότητα 15.

Η Scanner για κάθε θεμελιώδη τύπο διαθέτει μια συνάρτηση με την οποία μπορούμε να διαβάζουμε αντίστοιχα δεδομένα. Ο πίνακας 4.6 παρουσιάζει τις συναρτήσεις που μας ενδιαφέρουν ώστε να μπορούμε να υποστηρίζουμε στα προγράμματά μας είσοδο από το πληκτρολόγιο.

Τύπος επιστροφής	**Συνάρτηση**	**Χρήση**
String	next()	Είσοδος δεδομένων τύπου String
boolean	nextBoolean()	Είσοδος δεδομένων τύπου boolean
byte	nextByte()	Είσοδος δεδομένων τύπου byte
double	nextDouble()	Είσοδος δεδομένων τύπου double
float	nextFloat()	Είσοδος δεδομένων τύπου float
int	nextInt()	Είσοδος δεδομένων τύπου int
String	nextLine()	Είσοδος δεδομένων τύπου String
long	nextLong()	Είσοδος δεδομένων τύπου long
short	nextShort()	Είσοδος δεδομένων τύπου short

Πίνακας 4.6 *Συναρτήσεις της Scanner για είσοδο δεδομένων*

Σε κάθε κλήση τους, κάθε μια από τις συναρτήσεις του πίνακα 4.6 διαβάζει μια ολοκληρωμένη σειρά δεδομένων (token). Εξ ορισμού, μια σειρά δεδομένων θεωρείται ότι τερματίζεται από την παρουσία ενός διαστήματος (whitespace). Εξαίρεση αποτελεί η συνάρτηση nextLine που διαβάζει τα δεδομένα μιας γραμμής, θεωρώντας το διάστημα ως μέρος των δεδομένων.
Στον κώδικα 4.20 επιδεικνύεται ενδεικτική χρήση των συναρτήσεων εισόδου. Στη γραμμή 2, ορίζουμε το αντικείμενο in της κλάσης Scanner μέσα από το οποίο έχουμε τη δυνατότητα να καλούμε τις συναρτήσεις εισόδου. Κατά τον ορισμό του in, το συνδέουμε με την System.in. Με αυτόν τον τρόπο καθορίζεται πως η είσοδος θα γίνεται από το πληκτρολόγιο.

```
1     static void keyboardInput() { //Κώδικας 4.20
2        Scanner in = new Scanner(System.in);
3        System.out.print("Δώστε όνομα : ");
4        String name = in.nextLine();
5
6        System.out.print("Δώστε αριθμό Μητρώου: ");
7        int aem = in.nextInt();
8
9        System.out.print("Εισόδημα έτους 2021 : ");
10       double eisodima = in.nextDouble();
11       System.out.println();
12
13       System.out.println("Όνομα: " + name);
14       System.out.println("ΑΜ : " + aem);
15       System.out.println("Εισόδημα 2021: " + eisodima);
16      // in.close();
17    }
```

Κώδικας 4.20*: Είσοδος από το πληκτρολόγιο*

Στη γραμμή 4 διαβάζουμε μια γραμμή δεδομένων. Θα μπορούσαμε να χρησιμοποιήσουμε τη συνάρτηση next(), μιας και το όνομα είναι τύπου String. Ωστόσο, αν ο χρήστης καταχωρήσει όνομα και επώνυμο διαχωρισμένα με διάστημα, η next() θα διαβάσει μόνο μέχρι το διάστημα, δηλαδή μόνο το όνομα. Ακόμη χειρότερα, η επόμενη είσοδος που επιχειρείται με την nextInt στη γραμμή 7 θα βρει ως δεδομένα εισόδου το επώνυμο του χρήστη. Θα βρει δηλαδή ένα String, ενώ περιμένει int, οπότε θα παραχθεί λάθος χρόνου εκτέλεσης.

Εφόσον διαβάσουμε σωστά το ονοματεπώνυμο του χρήστη, προχωράμε στην είσοδο του αριθμού μητρώου. Αν εδώ ο χρήστης δεν δώσει ακέραια τιμή, επίσης θα παραχθεί λάθος χρόνου εκτέλεσης. Στη συνέχεια, στη γραμμή 10, διαβάζουμε έναν αριθμό τύπου double.

Το παράδειγμα κλείνει με την in.close() σχολιασμένη. Προσοχή, μην κλείσετε ποτέ ένα αντικείμενο Scanner που είναι σχετισμένο με την System.in. Αν το κάνετε, η εφαρμογή σας δεν θα είναι σε θέση να ανοίξει ροή επικοινωνίας με το πληκτρολόγιο παρά μόνο αν επανεκκινηθεί [3].

Βέβαια, η σωστή χρήση της Scanner απαιτεί τη διαχείριση των λαθών που ενδέχεται να παραχθούν, ζήτημα με το οποίο ασχολούμαστε στην ενότητα 16.

4.4 Ασκήσεις

1. Ποια είναι η έξοδος του κώδικα που ακολουθεί;

```
int i = 0, j = 0;
System.out.print(i++ + " " + (++j) + " ");
System.out.println(i == j);
```

2. Ποια είναι η έξοδος του κώδικα που ακολουθεί; Εξηγήστε την απάντησή σας.

```
int k = 2;
System.out.println((k += 2) == (k *= 2));
```

3. Ποια είναι η έξοδος του κώδικα;

```
int i,j,k;
i = j = k = 0;
System.out.println((i++ == 0 || j++ == 0) && k++ == 0);
System.out.println(i + " " + j + " " + k);
```

4. Ποια είναι η έξοδος του κώδικα;

```
int i,j,k;
i = j = k = 0;
System.out.println(i++ == 0 || j++ == 0 && k++ == 0);
System.out.println("i++ == 0 || j++ == 0 && k++ == 0");
System.out.println(i + " " + j + " " + k);
```

5. Ποια είναι η έξοδος του κώδικα;

```
int i,j,k;
i = j = k = 0;
System.out.println(i++ == 0 && j++ == 0 || k++ == 0);
System.out.println("i++ == 0 && j++ == 0 || k++ == 0");
System.out.println(i + " " + j + " " + k);
```